real-tr-p1-full

An AgentMPI run analysis

Abstract

This is the sequential baseline of the parallel book-translation strong-scaling series, and it exists to be a denominator. One rank translated all eight sections of the book in 576.74 s using sixteen agent calls. Because the communicator has one member, every collective in the harness degenerates: `scatterv`, both glossary `allreduce`s, the pre-gather `barrier` and the `gather` return without sending a message, and the harness's own phase timers record 0.000 s for all four. That makes this the one run in the campaign that measures the task without the protocol, and it lets AgentMPI's fixed overhead be measured directly rather than inferred. It is 58.585 s, 10.16% of the wall clock, and it decomposes sharply: 54.473 s is the broker waiting for a worker process to claim a published task (3.405 s per call), 4.071 s is the 0.5 s completion poll aliasing, and everything the protocol itself does — five collective invocations, sixteen content-addressed store writes, 106 logged events, sixteen contract validations — accounts for 40.3 ms, or 0.0070% of the run. The single most important thing to take from this document is that those two numbers should never be added together: AgentMPI's coordination machinery costs about 2.5 ms per agent call, and its task-dispatch mechanism costs 3.4 s per agent call, a ratio of 1,360 to one. The second most important is a caveat on the curve this run anchors. At $p = 1$ the harness skips the seam-reconciliation phase entirely, so the baseline performs 16 agent calls where $p = 8$ performs 24 and reconciles none of the book's seven section boundaries where $p = 8$ reconciles all seven. The denominator is cheaper than the numerators because it is doing less.

1 What this run is

property	value
run	`real-tr-p1-full`
experiment	`translation`
campaign label	`real-tr`
declared world size	1
ranks appearing in the log	8
executors	`broker` × 1, `worker` × 33
events	106
wall time	576.74 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.90×	total busy time over wall time
parallel efficiency	89.8%	achieved parallelism per rank
idle fraction	10.2%	rank-seconds paid for and unused
peak ranks busy	1	of 1 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	1.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	26	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	33.76 / 55.52 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	28,760 / 4,996	prompt and completion
cost	$0.1612	0.0323 $/kTok out

3 What the analysis notices

- [warning] Ranks 1, 2, 3, 4, 5, 6, 7 appear in the log without participating; the declared world size is 1. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- [warning] 5 collective(s) completed but recorded no duration (`allreduce`, `barrier`, `gather`, `scatter`), so they contribute nothing to the coordination figures even though every participant blocked in them. The reported coordination share is short by exactly their blocking; it can be recovered from the event timestamps.
- [warning] The cost model's α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -16.04\,\text{s}$), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.697$. Any latency prediction from this run's calibration is a median, not a model.
- [ok] 26 rank reattachment(s) occurred, up to incarnation 17: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 5 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	518.16	89.8	16	16	0	0	0	0.0	finalized

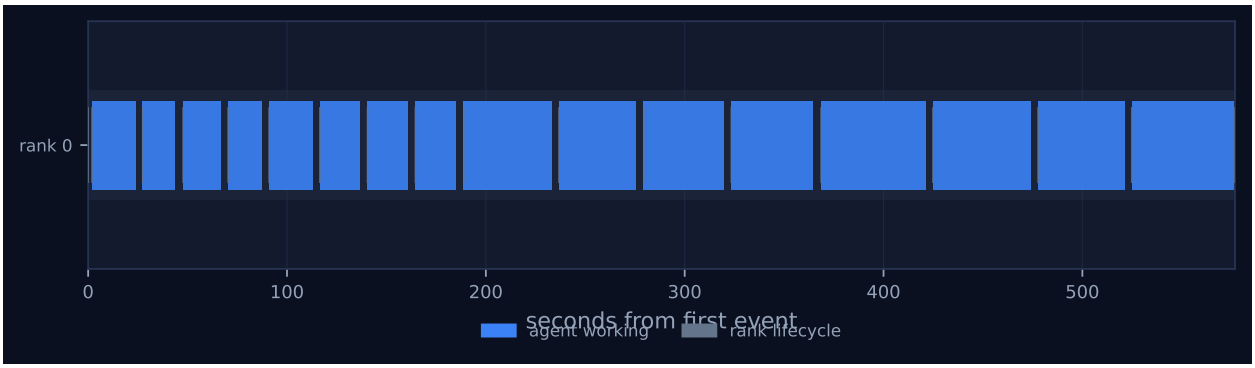

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

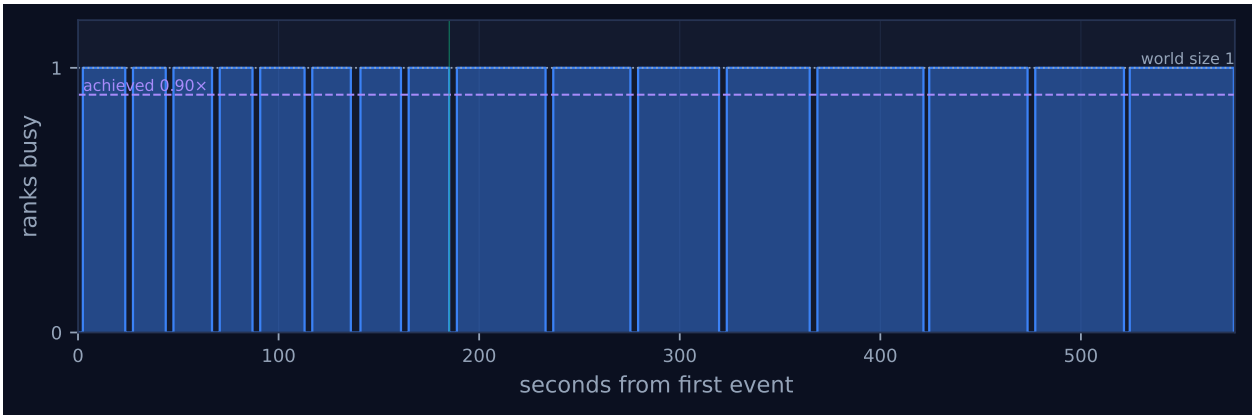

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

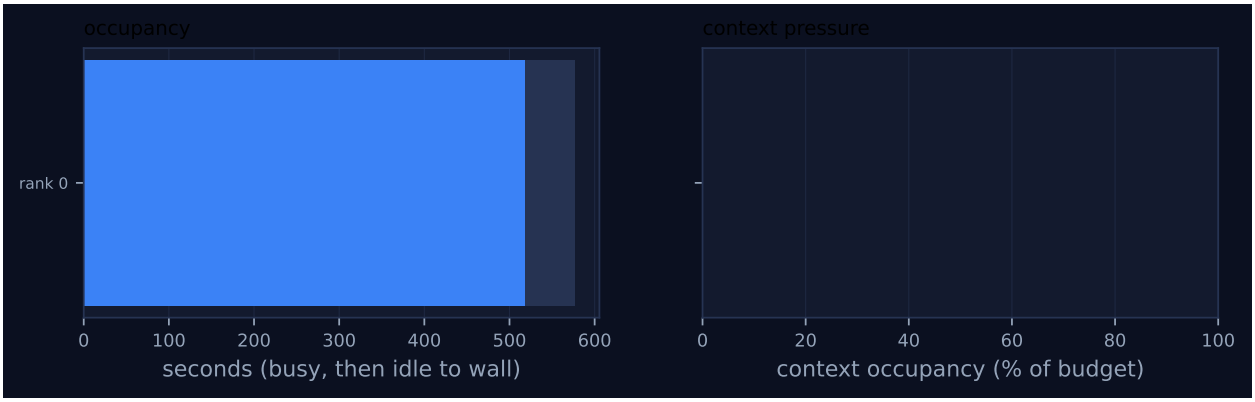

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

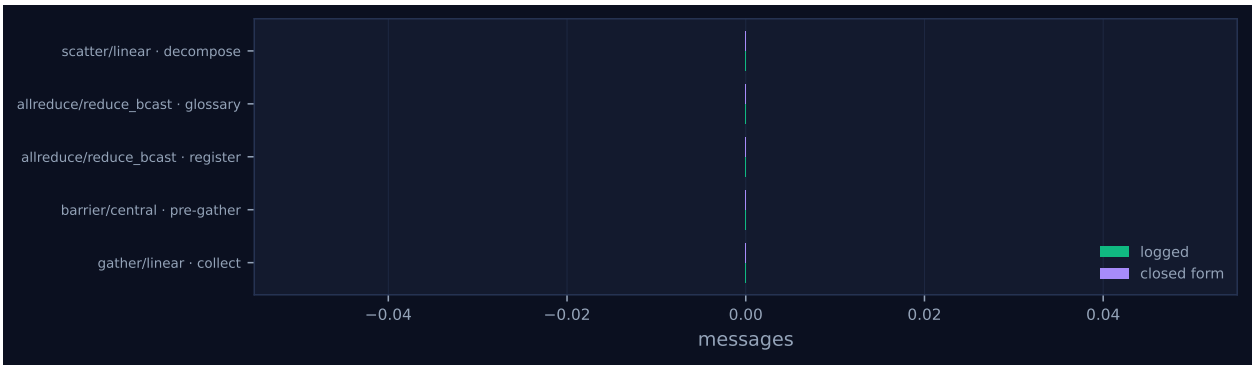

Figure 4: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	linear	decompose	1	1	0	0	0	0	0	0.00	0.00	✓
allreduce	reduce_bcast	glossary	1	1	0	0	0	0	0	0.00	0.00	✓
allreduce	reduce_bcast	register	1	1	0	0	0	0	0	0.00	0.00	✓
barrier	central	pre-gather	1	1	0	2	0	0	0	0.00	0.00	✓
gather	linear	collect	1	1	0	0	0	0	0	0.00	0.00	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is one lane with sixteen blue bars in it and nothing else, and the viewer screenshot shows the same picture with seven empty lanes underneath. There is no communication to draw: the run logged zero messages, so there are no green ticks anywhere, and the collectives panel in the dashboard prints *0* in every numeric column of all four of its rows. Figure 2 is consequently a square wave between one and zero, and Figure 4 has no bars at all, because a collective that sends no messages has nothing to compare against its closed form.

The interesting structure is therefore not in the bars but in the gaps between them, and they are remarkably regular. Each of the sixteen calls follows the same four-event cycle: `broker.enqueue`, `broker.claim`, `broker.complete`, `agent.call`. Measuring each interval across all sixteen cycles partitions the wall clock exactly:

interval	seconds	% of wall	what it is
`claim` → `complete`	518.156	89.844	the model working
`enqueue` → `claim`	54.473	9.445	waiting for a worker to take the task
`complete` → `agent.call`	4.071	0.706	the broker's 0.5 s completion poll
everything else	0.040	0.007	collectives, fabric writes, prompt construction
wall	576.741	100.000	

The four rows sum to 576.740 of 576.741 s; the missing millisecond is the interval between the last `rank.finalize` and `job.finish`. The 518.156 s in the first row is exactly the `busy_s` the headline table reports, and the 89.844% is its 89.8% occupancy figure.

Two features of that partition are worth reading off the picture rather than the table. The first is that the gaps are all the same size. Fifteen of the sixteen dispatch intervals lie between 3.086 and 3.923 s, with a median of 3.465 s; the sixteenth, the very first, is 1.945 s, and it is short because six worker processes had already registered against the job in the first two seconds before any task existed to claim. Every subsequent claim is preceded by a `rank.init` at the same millisecond, at incarnation $n + 1$ — the trace records a fresh process attaching to the rank and immediately taking the waiting task. Rank 0 reaches incarnation 17. So the uniform gap is not a queue and it is not contention; it is the cost of starting a new agent process for every invocation, and at $p = 1$ the whole of it lands on the critical path because there is nothing else to overlap it with.

The second is that the bars come in two clearly separated sizes, and Figure 1 shows the boundary: the first eight bars average 19.60 s and the last eight average 45.17 s. Those are the terminology-extraction and translation phases, and the transition happens at $t = 185.082$ s, where the harness records `extract` at 185.081 s and `glossary` at 0.000 s in the same breath. This bimodality is why the headline "agent latency p50 42.02 s" should not be read as a typical call. The sixteen latencies are eight values between 20.0 and 25.5 s and eight between 42.0 and 57.0 s, so the median falls in the empty gap between the two modes and describes no call in the run. It is also reported inconsistently: the headline table says 42.02 s and the per-rank profile and the calibration both say 33.764 s, from the same sixteen numbers. `cost.summarise` takes `lat[len(lat)//2]`, the ninth of sixteen, where `cost.calibrate` takes `statistics.median`, the mean of the eighth and ninth. On an even-sized bimodal sample the two conventions differ by 8.25 s, and this run is the starkest case in the campaign because its sample is exactly eight short calls and eight long ones.

Figure 3 has an empty right-hand panel: context occupancy is 0.0% against a 120,000-token budget, and `rank.finalize` reports `used: 0`, `high_water: 0`, `admissions: 0`. Nothing this run did touched an agent's context account, which is by construction — there is nothing to receive.

The last thing the picture shows is what the plot omits. Figure 1 draws one lane; the viewer draws eight, because ranks 1 through 7 registered against a world of size 1 and are each represented by a short row of grey lifecycle ticks scattered across the run. That is the subject of a later section.

7 Interpretation

The collectives really did degenerate

This is the premise the whole document rests on, so it is worth establishing from the trace rather than asserting from the world size. Three independent lines of evidence agree.

From the event log: the run contains five `coll.*` events and no `msg.send` or `msg.recv` at all. Every one of the five carries `messages_sent: 0`, `tokens_sent: 0`, `rounds: 0` and `wall_s: 0.0`. Notably the log contains *no* `coll.reduce` and *no* `coll.bcast`, where every $p > 1$ sibling contains two of each per `allreduce`. The `reduce_bcast` algorithm did not merely send zero messages, it never delegated.

From the source: `algorithms.allreduce` tests `if p == 1` at line 1052 and returns its payload before reaching the `reduce_bcast` branch on the next line. The same guard appears in `bcast`, `scatter`, `gather`, `allgather`, `reduce` and `scan`, and `barrier` tests `p <= 1`. In every case the body of the guard is three statements: finish the timer, call `_record`, return. `_record` is the fabric write and the event emit, and it is the entirety of what a single-rank collective does.

From the harness's own instrumentation: `per_rank_stats` for rank 0 reports `decompose: 0.0`, `glossary: 0.0`, `halo: 0.0` and `collect: 0.0` seconds, against `extract: 185.081` and `translate: 391.656`. Those timers round to three decimals, so each of the four coordination phases took under half a millisecond. The `halo` zero has a different cause from the other three — `cfg.halo and comm.size > 1` is false, so the phase was skipped rather than degenerated — and that distinction matters enough to have its own section below.

The residual: what AgentMPI costs when it has nothing to coordinate

The four-row partition in the previous section is the measurement, and the interesting row is the last one. Forty milliseconds, of a 576.741 s run, is everything AgentMPI did that was not waiting for a worker or waiting for a model. Broken down by where the intervals fall:

host-side interval	occurrences	total (ms)
`agent.call` → next `broker.enqueue`	14	33.6
`agent.call` → `broker.enqueue` across the glossary phase	1	4.4
`job.create` → `coll.scatter` → first `broker.enqueue`	1	2.7
last `agent.call` → `barrier` → `gather` → `finalize` → `job.finish`	1	0.6
total		40.3

The dominant row is the enqueue path, and it is 2.40 ms on average over the fourteen clean samples. That interval contains: parsing and contract-checking the previous result, building the next prompt, writing it to the content-addressed blob store, an `INSERT` into `agent_calls` inside a SQLite write transaction, and the `broker.enqueue` emit inside the same transaction. Sixteen of those is 38.4 ms, which is essentially the whole 40.3.

The collectives are visible in this table only as the difference between 4.4 ms and the 2.40 ms that an ordinary enqueue costs, because the two glossary `allreduce`s and the glossary blob write both fall inside that one interval. Individually, the successive event timestamps bound each collective's own cost: 0.138 ms between the last `terms agent.call` and the first `coll.allreduce`, 0.078 ms between the two `allreduce`s, 0.126 ms between the last `agent.call` and `coll.barrier`, 0.051 ms between `coll.barrier` and `coll.gather`, and 0.113 ms from there to `rank.finalize`. A degenerate collective costs between 0.05 and 0.14 ms, and all five together cost under a millisecond. Against 106 logged events the whole residual works out at 0.38 ms per event, which is the right order for one SQLite row insert with the fabric's default synchronous settings.

The completion-poll row, 4.071 s, is a different kind of cost and is worth separating from the other

two. `BrokerExecutor.poll_s` is 0.5, so a task that finishes is noticed on average 0.25 s later. The sixteen observed lags average 0.2545 s and none exceeds 0.471 s, which is what pure aliasing against a 0.5 s grid predicts; the 4.5 ms of excess over the expected 0.250 s upper-bounds the cost of reading the result blob, validating it against the contract and emitting `agent.call`. Sixteen samples is too few to defend 4.5 ms as an estimate, but it is a sound bound, and it is consistent with the 2.40 ms measured on the enqueue side. The same aliasing is visible in the reported latencies themselves: all sixteen `latency_s` values lie within 21 ms of a multiple of 0.5 s.

So the honest statement of AgentMPI's fixed overhead at $p = 1$ is two numbers, not one. The *protocol* — collectives, fabric persistence, event logging, contract validation — costs about 2.5 ms per agent call and 0.007% of this run. The *dispatch mechanism* costs 3.405 s per agent call and 9.45% of this run. Only the first is a property of AgentMPI's design; the second is a property of running one ephemeral worker process per invocation, and the trace shows it plainly, because every `broker.claim` in the run is preceded at the same millisecond by a `rank.init` at a new incarnation.

The two halves of that 3.4 s can be separated using the rest of the series. Pooling all 80 dispatch intervals across the four completing points and splitting them by whether a `rank.init` for that rank falls inside the interval, the 64 that started a fresh worker average 3.075 s and the 16 that were taken by a process already attached and idle average 1.012 s. Process start-up is therefore about 2.06 s and the claim path about 1.0 s. At $p = 1$ all sixteen dispatches started a fresh worker — the worst case available — which is why this run's per-call dispatch is the highest in the series. It is also not constant across the series. Dispatch per call is 3.405 s at $p = 1$, 3.412 s at $p = 2$, 2.633 s at $p = 4$ and 1.632 s at $p = 8$, and because at $p > 1$ it overlaps across ranks, the amount of it that lands on the critical path falls roughly as 54.5, 30.7, 14.5, 4.9 s. Some of the measured speedup in the paper's table is this fixed per-call cost amortising, and I quantify that below.

What this run contributes to the scaling curve

Wall times across the four completing points are 576.74, 410.11, 308.44 and 166.78 s, giving

p	wall (s)	speedup	efficiency	Karp–Flatt	calls	model rank-s	achieved par. eff.
1	576.74	1.000	1.000	—	16	518.16	0.898
2	410.11	1.406	0.703	0.422	18	670.53	0.818
4	308.44	1.870	0.467	0.380	22	906.20	0.735
8	166.78	3.458	0.432	0.188	24	1,033.19	0.774

The efficiency column falls from 1.00 to 0.43, and the last two columns say where it went. Write $W(p)$ for the model rank-seconds — the sum of `claim`-to-`complete` intervals — and $E_{\text{par}}(p) = W(p)/(p\,T_p)$ for the achieved parallel efficiency in the last column. Then end-to-end efficiency factorises exactly:

$$E(p) \;=\; \frac{T_1}{p\,T_p} \;=\; \underbrace{\frac{T_1}{W(1)}}_{1.1131} \;\times\; \underbrace{\frac{W(1)}{W(p)}}_{1/A(p)} \;\times\; E_{\text{par}}(p),$$

and it closes to three decimals at every point: $1.1131 \times 0.7728 \times 0.8175 = 0.7031$ at $p = 2$, $1.1131 \times 0.5718 \times 0.7345 = 0.4675$ at $p = 4$, and $1.1131 \times 0.5015 \times 0.7743 = 0.4322$ at $p = 8$.

Three things follow. First, the leading constant is this run's own overhead: because the baseline spends 11.31% more wall clock than model time, every speedup in the table is inflated by a factor

of 1.1131 relative to the model work actually done. Second, the work-amplification term $1/A(p)$ is the largest single contributor to the loss at $p = 8$: the parallel run performs $A(8) = 1.994$ times the model rank-seconds of the baseline. Third, once that is divided out, what remains — E_{par}, which is genuinely about waiting — is 0.818, 0.735 and 0.774, and it is *not* monotone in p.

Doing the same division on the speedup gives $S_w(p) = S(p)\,A(p) = 1.000$, 1.820, 3.270, 6.895, and fitting the USL to those four points instead of the wall-clock ones gives $\sigma = 0.025$, $\kappa = 0.0000$, $R^2 = 0.989$, against the paper's $\sigma = 0.220$, $\kappa = 0.0000$, $R^2 = 0.873$. I computed both with the repository's own `agentmpi.cost.fit_usl` and the second reproduces the paper's macro exactly. The reading is that roughly nine tenths of the $\sigma = 0.220$ serial fraction the paper reports is not coordination at all: it is the parallel runs doing more work than the baseline they are divided by.

Does the Karp–Flatt column support the paper's claim?

Not as stated, and the column is the right place to see why. Karp–Flatt is constructed so that a *constant* value across p is the signature of a fixed serial section, and an *increasing* value the signature of a coordination cost that grows with p — the docstring in `agentmpi.cost.karp_flatt` says exactly this. The measured column is 0.422, 0.380, 0.188: monotone decreasing, and down by 55% from $p = 2$ to $p = 8$.

That rules out the ceiling. A coherency term would push the metric up, and the USL's $\kappa = 0.0000$ agrees; nothing in $p \le 8$ shows a retrograde region. But it equally rules out the constant serial fraction that $\sigma = 0.220$ asserts. The empirical serial fraction is 0.422 at $p = 2$ and 0.188 at $p = 8$, and 0.220 lies between them without matching either. The residuals of the fit confirm this is systematic rather than noise: the USL over-predicts speedup at $p = 2$ and $p = 4$ by 0.233 and 0.540 and under-predicts at $p = 8$ by 0.308, a sign pattern of $-, -, +$ rather than a scatter. An R^2 of 0.873 from a two-parameter monotone curve fitted to four monotone points is weak evidence for the functional form, and it is worth saying that the fit was only reportable at all after the $p = 16$ attempt was dropped; with it, the same grid search returns $\sigma = 0.020$, $\kappa = 0.0465$, $R^2 = 0.404$, which `make_tables.py` suppresses because it falls below its $R^2 \ge 0.5$ threshold.

Decreasing Karp–Flatt is the signature of a fixed cost being amortised, and this run supplies two candidates for it. The work amplification above is one: dividing it out drops the Karp–Flatt column to 0.099, 0.074, 0.023 and lifts R^2 to 0.989. Broker dispatch is the other: the baseline pays 54.5 s of it on the critical path and $p = 8$ pays about 4.9 s, and subtracting the critical-path estimate at each point gives speedups of 1.000, 1.377, 1.777 and 3.226 — lower than the reported ones, confirming that the dispatch cost inflates the curve rather than depressing it. Both are fixed per-call costs, both amortise as p grows, and neither is a serial fraction of the algorithm.

This baseline is not running the same experiment

The harness guards the seam-reconciliation phase with `if cfg.halo and comm.size > 1`, so at $p = 1$ there is no `cart_create`, no `halo_exchange` and no `revise:*` call. The trace confirms it: 16 agent calls, eight `terms` and eight `translate`, and `n_boundary_changes: 0`.

The consequence is structural, not incidental. The book is cut into eight units, so it has seven internal seams, and the halo exchange reconciles exactly those seams that fall on a rank boundary. At $p = 1$ that is none of them; at $p = 2$, one; at $p = 4$, three; at $p = 8$, all seven. The number of revision agent calls follows: 0, 2, 6, 8, which is $2(p - 1)$ while a rank holds more than one unit and p once each rank holds exactly one. That accounts for the whole of the call-count column in the

scaling table.

I checked whether the extra calls changed anything, by pulling every `translate:u`*n* and `revise:u`*n* result out of each run's blob store and diffing the `translation` strings. At $p = 2$ both revisions returned text byte-identical to the pre-revision translation: 0 of 2,437 characters moved. At $p = 4$, three of six revisions changed the text, 22 of 6,959 characters. At $p = 8$, all eight did, 68 of 9,527 characters. So the mechanism the baseline omits is close to inert at $p = 2$ and does progressively more as p grows, which is what one would expect when the fraction of seams it can see rises from 1/7 to 7/7.

None of this is visible in the quality metrics. Coverage, consistency, rendering verification and paragraph fidelity all read 1.000 at every p including this one, and `n_divergent_entities` is 0 everywhere. Whether the extra 1.994× of model work at $p = 8$ bought anything is therefore a question this instrument cannot answer, and the two defensible speedup numbers — 3.46× on wall clock, 6.90× on work — bracket the two possible answers. A reader who believes the seam revision is worth doing should quote the second; a reader who does not should quote the first and note that $p = 1$ is the only configuration that gets the cheaper artefact for free.

The calibration warning is right about the conclusion and wrong about the reason

The generated findings warn that α and β here are a median fallback rather than a fit, "because the regression returned a negative slope ($\beta = 0.1668$ s/token)". The conclusion is correct and worth heeding — the published $\alpha = 33.764$ s and $\beta = 0.053851$ s/token are robust medians and not a model — but the stated reason is not what happened, and the same wrong reason is printed on three of the four runs in this series.

Refitting the sixteen invocations by hand reproduces the regression's coefficients exactly: $\beta = +0.166796$ s/token, which is a positive slope of about six output tokens per second, and $\alpha = -16.04$ s, with $R^2 = 0.697$. The rejection test in `cost.calibrate` is `if beta > 0 and alpha > 0`, so what failed was the intercept, not the slope. The finding text in `scripts/analyze_run.py` hard-codes the phrase "negative slope" for every `median_fallback` case and then explains it with a queueing story — the broker wait entering the latency but not the token count — which would indeed produce a negative slope, and which is the diagnosis the docstring in `cost.py` records from the semantic-glossary run. Neither applies here. Broker queueing is not inside `latency_s` in this run at all: the 3.4 s dispatch wait lies between `enqueue` and `claim`, and the reported latency begins at the claim. What the fit is actually saying is that these calls are pure generation at a near-constant rate with no measurable fixed cost, so the honest model is $\alpha \approx 0$ with $\beta \approx 1/6$, and least squares on a narrow output range (209–472 tokens) pushes the intercept slightly below zero to buy a marginally better slope.

I judge this a real, if small, defect with two halves. The message is wrong about which coefficient was rejected. More substantively, the guard's $\alpha > 0$ condition rejects a physically sensible zero-intercept fit, and it does so selectively in a way that inverts the quality ordering across this series: $p = 1$, $p = 2$ and $p = 4$ are rejected with R^2 of 0.697, 0.612 and 0.624, while $p = 8$ is accepted as `least_squares` with $R^2 = 0.250$. The one published regression in the series is the worst of the four.

Seven stray ranks in a world of one

The generated findings flag ranks 1 through 7 as appearing without participating. They are the worst instance of this leak in the campaign, in ratio terms: the fabric's `ranks` table ends the run

with eight rows for a declared `world_size` of 1, so 87.5% of the recorded population is spurious, against 20% at $p = 8$.

The mechanism is the one the sibling analyses describe and the evidence here is consistent with it. `create_job` writes `world_size` into `meta` and populates `comm_members` with exactly one row; `RankRuntime.register` consults neither and performs an unconditional `INSERT` for whatever `wrank` the process was started with. Seven worker processes from the shared pool did so, in the first 4.6 s of the run, and seven more re-registered later — ranks 4, 5, 6 and 7 reach incarnation 2 within the first 23 s, ranks 1, 2 and 3 reach incarnation 3 as late as $t = 496.5$ s. Ten of the run's 26 reported reattachments are these; the other sixteen are rank 0's genuine one-per-call turnover.

What makes this run's version of the leak informative is the index set. The strays are exactly $1 \ldots 7$, which is the complement of a one-rank world inside an eight-rank one, and the $p = 8$ run of the same campaign finished 25 minutes earlier. The pool that leaked into this job is the pool that had just served `real-tr-p8-full`. The same reading holds across the series: `real-tr-p2-full` carries strays $2 \ldots 8$ and 14, `real-tr-p4-full` carries $4 \ldots 7$, 8 and 14, and `real-tr-p8-full` carries 8 and 14. One pool, indices 0–8 and 14, attaching to whichever job was live.

Containment held here as it did at $p = 8$: collectives resolve membership through `comm_members`, which has one row, so no tree ever addressed a stray and no `absent` list ever mentioned one. The residue is in the fabric — seven rows left in state `running`, with leases renewed to thirty minutes past a job that has already emitted `job.finish`. I count this as a genuine defect in `RankRuntime.register` rather than as untidiness, for the reason the $p = 8$ analysis gives: registration is an upsert keyed on rank index, so the only thing that kept these seven processes from taking rank 0's lease and mailbox is that their indices happened to be out of range. A bounds check against the `world_size` meta key would refuse all seven.

The glossary, and a code path that never ran

Rank 0's `allreduce` contributed 208 local terms and received 208, which is the identity it must be. Worth noting is that this is not a smaller glossary than the distributed runs produce: $p = 2$ agreed 207 terms, $p = 4$ agreed 208 and $p = 8$ agreed 206. The union of eight term sheets is the union of eight term sheets whether one agent or eight produce them, and the two-term spread is agent nondeterminism in what each sheet proposed, not a protocol effect. The `UNION` operator's claim to exactness is not tested by this run; it is tested by the sibling runs, and it holds there.

One consequence is that the harness's conflict-resolution rule — `sorted(v)[0]` over a term with multiple proposed renderings, the thing that guarantees every rank resolves identically — is dead code here. `n_conflicting_terms` is 0 at $p = 1$, and it is also 0 at $p = 2$ and $p = 4$; only at $p = 8$, where eight independent extractors each see one section, does it reach 4. The determinism argument the harness's docstring makes for choosing an exact operator is sound, but this run cannot support it and neither can two of its three siblings.

8 Threats to this reading

The 3.405 s dispatch figure measures the driver, not AgentMPI, and I cannot fully separate them. What the trace shows is the interval between an `agent_calls` row reaching state `pending` and a worker claiming it. Whether that 3.4 s is Cursor subagent spawn latency, an external scheduler's own poll, or something in `ampi worker next` is not visible in the log, and the fact that

it falls to 1.632 s per call at $p = 8$ — where eight workers are being spawned concurrently — suggests it is dominated by something with fixed start-up cost and spare capacity, which is an inference. What is measurement is that it is 9.45% of this run and that the protocol residual it is often lumped with is three orders of magnitude smaller.

The 40.3 ms residual is a lower bound on the protocol's cost, not the whole of it. It counts only work that happens on the broker thread between two of its own events. Fabric writes performed inside the claim and complete transactions are attributed to dispatch and to model work respectively, because the trace cannot see inside those intervals. The 33 worker registrations each write a `ranks` row and emit an event, and those writes are invisible here. A microbenchmark of the fabric would settle this; the trace cannot.

Sixteen samples, one draw, and the interesting quantities are means of them. The 2.40 ms enqueue cost, the 0.2545 s poll lag, the 3.405 s dispatch and the 45.17 s mean translate latency are all means over eight or sixteen observations from a single run. The dispatch figure in particular has a range of 1.945 to 3.923 s. Nothing in this document should be read to more precision than the sample supports, and the three-decimal figures are reported for reproducibility against the log, not as claims of accuracy.

The executor is a broker fronting real workers, but the log cannot see what ran inside one. `metrics.json` reports `executors: {broker: 1, worker: 33}`, so this is not a `simulated` or `function` run, and the outputs are real Chinese prose whose per-unit content varies in ways the deterministic `synthetic_agent` could not produce. But the quantisation of every latency to a 0.5 s grid is exactly what a scripted executor would also produce, and the only reason I read it as the broker's poll interval rather than as evidence of one is that `poll_s` is 0.5 in the source and no observed lag exceeds it.

This run ran last in the series and shares outputs with its predecessors, and the evidence does not support the campaign's usual reading of that. The campaign executed in descending order of p: $p = 8$ at 06:54, $p = 4$ at 07:03, $p = 2$ at 07:10, $p = 1$ at 07:19, all against one persistent worker pool. Four of this run's eight output sections are byte-identical to `real-tr-p2-full`'s, two of those are identical to `real-tr-p4-full`'s, and one — section 0, 1,371 characters — is identical to `real-tr-p8-full`'s, although *that* run's own unrevised translation of section 0 is different and it reached the shared text only after its `revise:u0` call moved seven characters. The contract asks for both explanations, and the usual verdict elsewhere in this campaign is convergence: a binding 206-term glossary, a pinned paragraph count and a fixed register leave a near-deterministic decoder little room, so byte-identity is a short step. That argument can be tested directly here, and it fails.

The test is the terminology phase. `prompt_terms` does not take the glossary, so a `terms:u`*`n`* prompt is a pure function of the unit, and all four runs issue *byte-identical* terms prompts — 8 of 8 prompt digests match across $p = 1, 2, 4, 8$. If the executor were near-deterministic, all 32 results would agree unit by unit. They do not. `terms:u4` carries prompt digest `ea002f41` in every run and returns four different answers: 36, 38, 38 and 34 terms, in 913, 956, 956 and 872 bytes, with four differently worded register sentences. Only 6 of the 48 cross-run terms pairs agree; the other 42 differ. Identical input therefore does not give identical output in this campaign, and a 1,300-character byte-identity cannot be attributed to determinism.

What the identities do track is placement and adjacency. Over the four runs' 48 unit-pairs of final output, all 11 pairs in which the unit sat on rank 0 in both runs are byte-identical and none of the other 37 are. This run's rank 0 reproduces all four of $p = 2$'s rank-0 terms results and none of its rank-1 results; $p = 2$ reproduces one of $p = 4$'s two rank-0 results; $p = 4$ reproduces none of $p = 8$'s. Reuse confined to one pool slot and decaying with distance in time is what that looks like;

convergence is not, because convergence would not care which rank index held the unit. Two caveats keep this short of proof: rank 0 always holds the lowest-numbered units in a contiguous split, so "same rank" and "early unit" cannot be separated within this series, and the identical answers were evidently generated rather than served from a cache — this run's `terms:u0` reports 24.01 s for its 337 output tokens, in line with every other call, where a cache hit would have returned at once. I read the balance of evidence as dependence between runs rather than convergence, and the practical consequence is bounded: the timings, event counts and message counts this document rests on are unaffected, but the quality column of the scaling table is not four independent draws.

This point is the denominator, and it is the least representative run in the series. It is the only one whose collectives are all degenerate, the only one with no messages, the only one with no seam reconciliation, and the only one where the entire dispatch cost is serialised onto the critical path. Every one of those makes it a good instrument for measuring fixed overhead and a questionable instrument for measuring what a one-rank version of the parallel program would cost. The speedups computed against it inherit all four properties.